Writing recipes - basics

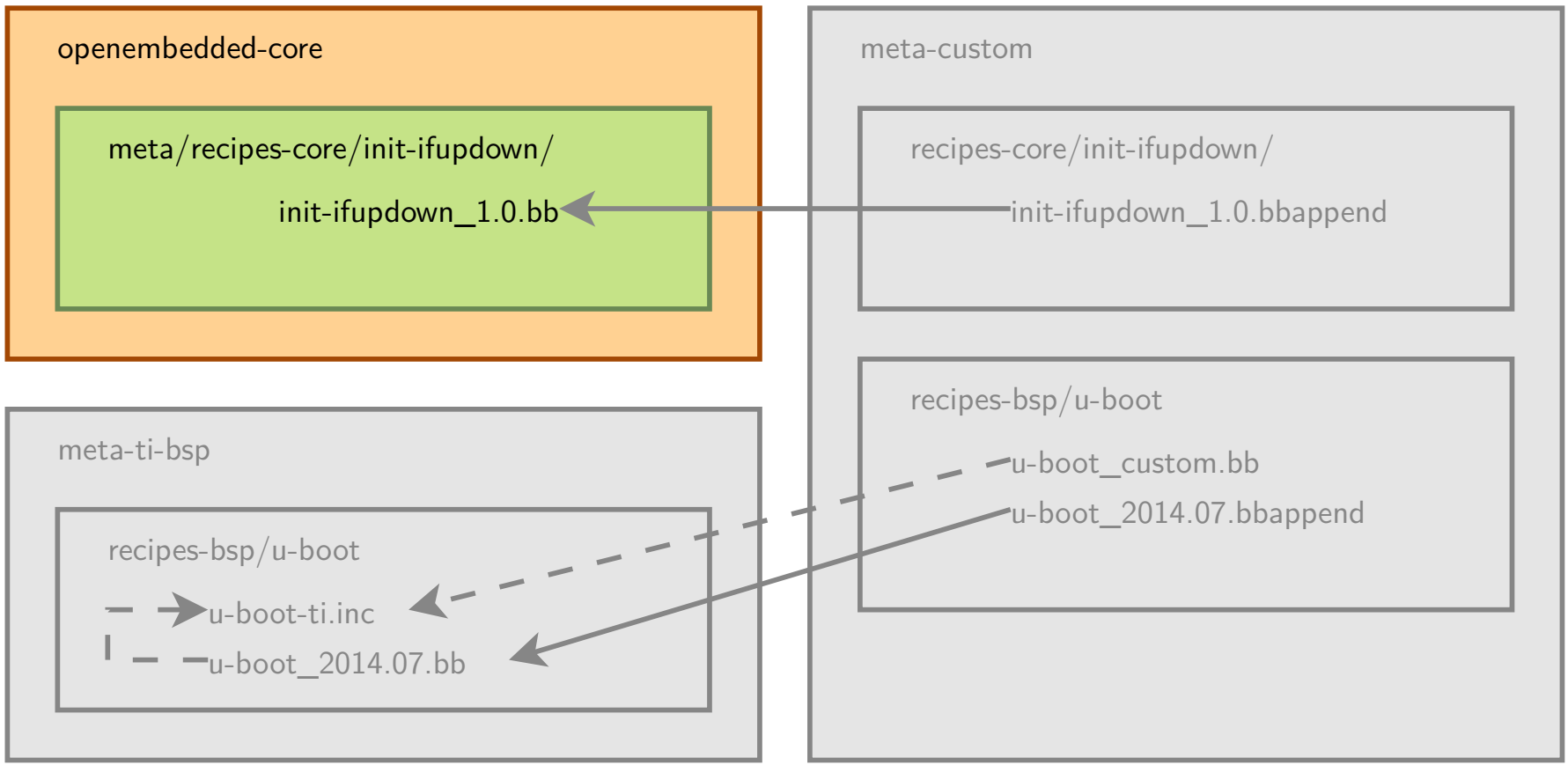


© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Recipes: overview

Recipes


openembedded-core
meta/recipes-core/init-ifupdown/
init-ifupdown_1.0.bb
meta-custom
recipes-core/init-ifupdown/
init-ifupdown_1.0.bbappend
recipes-bsp/u-boot
u-boot_custom.bb
u-boot_2014.07.bbappend
meta-ti-bsp
recipes-bsp/u-boot
u-boot-ti.inc
u-boot_2014.07.bb
extend
include/require



Basics

- A recipe describes how to handle a given software component (application, library, ...).
- It is a set of instructions to describe how to retrieve, patch, compile, install and generate binary packages.
- It also defines what build or runtime dependencies are required.
- Recipes are parsed by the `bitbake` build engine.
- The format of a recipe file name is `<recipename>_<version>.bb`
- The output product of a recipe is a set of binary packages (rpm, deb or ipk): typically `<recipename>`, `<recipename>-doc`, `<recipename>-dbg` etc.



Content of a recipe

- A recipe contains configuration variables: name, license, dependencies, path to retrieve the source code...
- It also contains functions that can be run (fetch, configure, compile...) which are called **tasks**.
- Tasks provide a set of actions to perform.
- Remember the `bitbake -c <task> <target>` command?



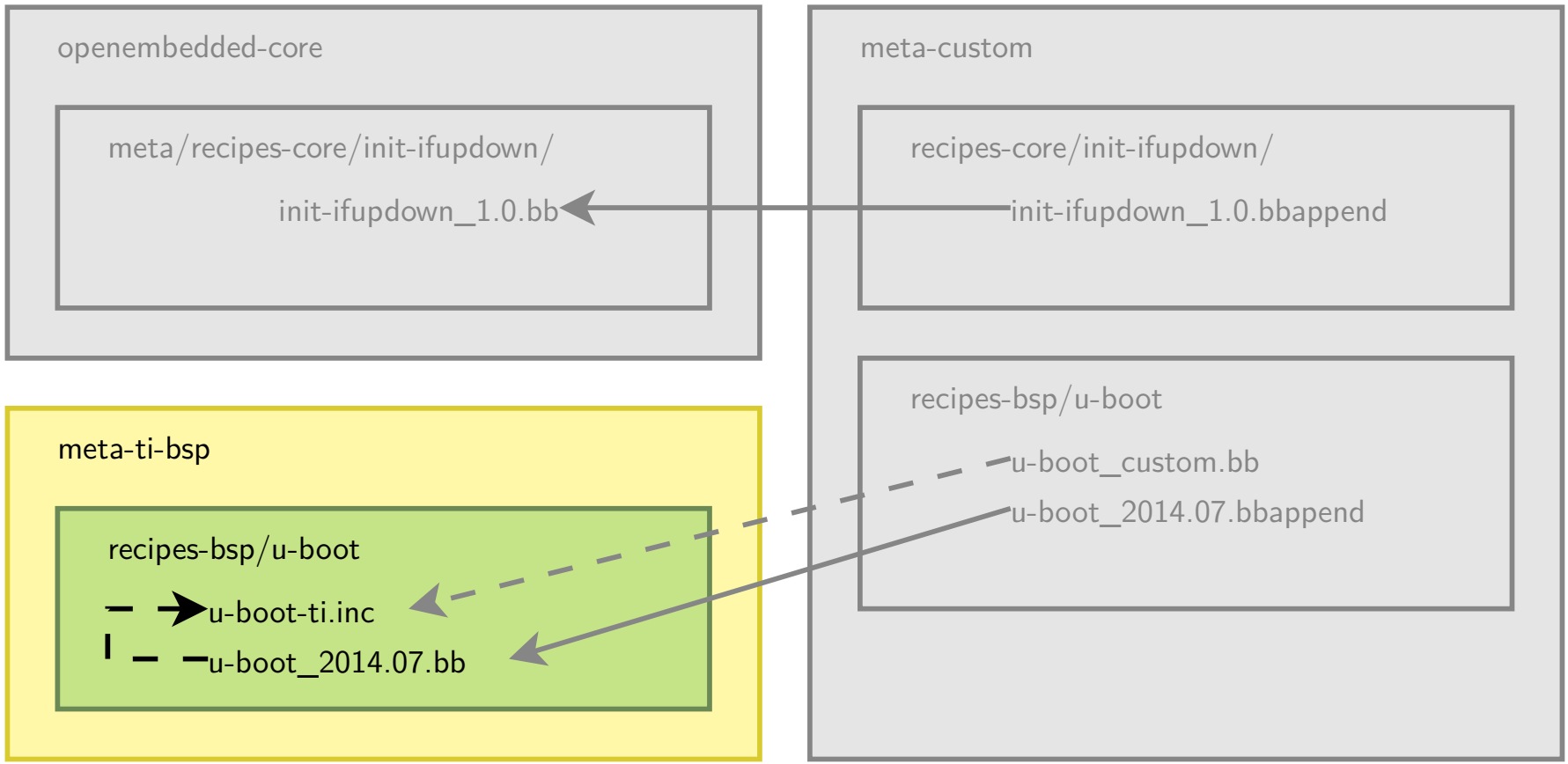
Common variables

- To make it easier to write a recipe, some variables are automatically available:
  - BPN: recipe name, extracted from the recipe file name
  - PN: BPN potentially with prefixes or suffixes added such as `nativesdk-`, or `-native`
  - PV: package version, extracted from the recipe file name
  - BP: defined as `${BPN}-${PV}`
- The recipe name and version usually match the upstream ones.
- When using the recipe `bash_5.1.bb`:
  - `${BPN} = "bash"`
  - `${PV} = "5.1"`



Organization of a recipe

Organization of a recipe





Organization of a recipe

- Many applications have more than one recipe, to support different versions. In that case the common metadata is included in each version specific recipe and is in a `.inc` file:
  - `<application>.inc`
    - version agnostic metadata
  - `<application>_<version>.bb`
    - `require <application>.inc`
    - any version specific metadata
- We can divide a recipe into three main parts:
  - The header: what/who
  - The sources: where
  - The tasks: how



The header

Configuration variables to describe the application:

- SUMMARY: short description for the package manager
- DESCRIPTION: describes what the software is about
- HOMEPAGE: URL to the project's homepage
- SECTION: package category (e.g. `console/utils`)
- LICENSE: the application's license, using SPDX identifiers (https://spdx.org/licenses/)



The source locations: overview

- We need to retrieve both the raw sources from an official location and the resources needed to configure, patch or install the application.
- SRC_URI defines where and how to retrieve the needed elements. It is a set of URI schemes pointing to the resource locations (local or remote).
- URI scheme syntax: `scheme://url;param1;param2`
- `scheme` can describe a local file using `file://` or remote locations with `https://`, `git://`, `svn://`, `hg://`, `ftp://`...
- By default, sources are fetched in `$BUILDDIR/downloads`. Change it with the DL_DIR variable in `conf/local.conf`



The source locations: remote files 1/2

- The `http`, `https` and `ftp` schemes:
  - `https://example.com/application-1.0.tar.bz2`
  - A few variables are available to help pointing to remote locations: `${SOURCEFORGE_MIRROR}`, `${GNU_MIRROR}`, `${KERNELORG_MIRROR}`...
  - Example: `${SOURCEFORGE_MIRROR}/<project-name>/${BPN}-${PV}.tar.gz`
  - See `meta/conf/bitbake.conf`
- The `git` scheme:
  - `git://<url>;protocol=<protocol>;branch=<branch>`
  - When using git, it is necessary to also define SRCREV. It has to be a commit hash and not a tag to be able to do offline builds (a git tag can change, you then need to connect to the repository to check for a possible update). The `branch` parameter is mandatory as a safety check that SRCREV is on the expected branch.



The source locations: remote files 2/2

- A checksum must be provided when the protocol used to retrieve the file(s) does not guarantee their integrity. This is the case for `https`, `http` or `ftp`.

```
SRC_URI[sha256sum] = "5891b5b522d..."
```

- It's possible to use checksums for more than one file, using the `name` parameter:

```
SRC_URI = "http://example.com/src.tar.bz2;name=tarball \
           http://example.com/fixes.patch;name=patch"

SRC_URI[tarball.sha256sum] = "97b2c3fb082241ab5c56..."
SRC_URI[patch.sha256sum] = "b184acf9eb39df794ffd..."
```



The source locations: local files

- SRC_URI items using the `file://` scheme are *local files*
- They are not downloaded, but rather copied from the layer to the work directory
- The searched paths are defined in the FILESPATH variable
- FILESPATH is a colon-separated list of paths to look for files
- The order matters: when a file is found in a path, the search ends



FILESPATH 1/3

- FILESPATH is generated with all combinations of:
- Base paths
  - `${FILE_DIRNAME}/${BP}` (e.g. `BP` = `dropbear-2020.81`)
  - `${FILE_DIRNAME}/${BPN}` (e.g. `BPN` = `dropbear`)
  - `${FILE_DIRNAME}/files`
  - Items in FILESEXTRAPATHS (none by default)
  - `${FILE_DIRNAME}` is the directory containing the `.bb` file
- The overrides in FILESOVERRIDES
  - Set as `${TRANSLATED_TARGET_ARCH}:${MACHINEOVERRIDES}:${DISTROOVERRIDES}`
  - E.g. `arm:armv7a:ti-soc:ti33x:beaglebone:poky`
  - Applied right to left



FILESPATH 2/3

- This results in a long list, including:
  - /.../meta/recipes-core/dropbear/dropbear-2020.81/poky/
  - /.../meta/recipes-core/dropbear/dropbear/poky/
  - /.../meta/recipes-core/dropbear/files/poky/
  - /.../meta/recipes-core/dropbear/dropbear-2020.81/beaglebone/
  - /.../meta/recipes-core/dropbear/dropbear/beaglebone/
  - /.../meta/recipes-core/dropbear/files/beaglebone/
  - /.../meta/recipes-core/dropbear/dropbear-2020.81/ti33x/
  - /.../meta/recipes-core/dropbear/dropbear/ti33x/
  - /.../meta/recipes-core/dropbear/files/ti33x/
  - ...
  - /.../meta/recipes-core/dropbear/dropbear-2020.81/armv7a/
  - /.../meta/recipes-core/dropbear/dropbear/armv7a/
  - /.../meta/recipes-core/dropbear/files/armv7a/
  - ...
  - /.../meta/recipes-core/dropbear/dropbear-2020.81/
  - /.../meta/recipes-core/dropbear/dropbear/
  - /.../meta/recipes-core/dropbear/files/



FILESPATH 3/3

- This complex logic allows to use different files without conditional code
- Example: with a single item in SRC_URI:

```
SRC_URI += "file://defconfig"
```

a different `defconfig` can be used for different MACHINE values:

```
recipes-kernel/
└── linux/
    ├── my-linux/
    │   ├── mymachine1/
    │   │   └── defconfig <-- used when MACHINE="mymachine1"
    │   ├── mymachine2/
    │   │   └── defconfig <-- used when MACHINE="mymachine2"
    │   └── defconfig     <-- used for any other MACHINE value
    └── my-linux_6.4.bb
```



The source locations: tarballs

- When extracting a tarball, `bitbake` expects to find the extracted files in a directory named `<application>-<version>`. This is controlled by the S variable. If the directory has another name, you must explicitly define S.



The source locations: license files

- License files must have their own checksum.
- LIC_FILES_CHKSUM defines the URI pointing to the license file in the source code as well as its checksum.

```
LIC_FILES_CHKSUM = "file://gpl.txt;md5=393a5ca..."
LIC_FILES_CHKSUM = "file://main.c;beginline=3;endline=21;md5=58e..."
LIC_FILES_CHKSUM = "file://${COMMON_LICENSE_DIR}/MIT;md5=083..."
```

- This allows to track any license update: if the license changes, the build will trigger a failure as the checksum won't be valid anymore.



Dependencies 1/2

- A recipe can have dependencies during the build or at runtime. To reflect these requirements in the recipe, two variables are used:
  - DEPENDS: List of the recipe build-time dependencies.
  - RDEPENDS: List of the package runtime dependencies. Must be package specific (e.g. with `:${PN}`).
- `DEPENDS = "recipe-b"`: the local `do_prepare_recipe_sysroot` task depends on the `do_populate_sysroot` task of recipe-b.
- `RDEPENDS:${PN} = "package-b"`: the local `do_build` task depends on the `do_package_write_<archive-format>` task of recipe-b.



Dependencies 2/2

- Sometimes a recipe has dependencies on specific versions of another recipe.
- `bitbake` allows to reflect this by using:
  - `RDEPENDS:${PN} = "recipe-b (>= 1.2)"`
- The following operators are supported: `=`, `>`, `<`, `>=` and `<=`.



Tasks

Default tasks already exist, they are defined in classes:

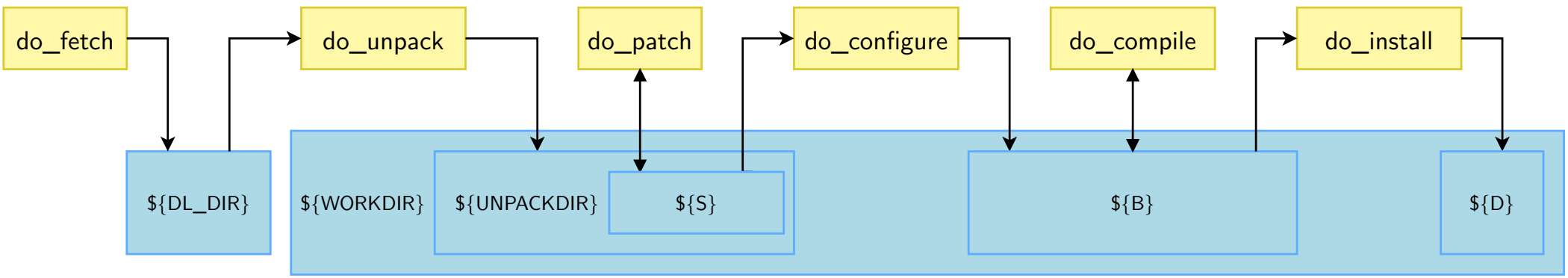
- do_fetch
- do_unpack
- do_patch
- do_configure
- do_compile
- do_install
- do_package
- do_rootfs

You can get a list of existing tasks for a recipe with:

```
bitbake <recipe> -c listtasks
```



The main tasks





Writing tasks 1/2

- Syntax of a task:

```
do_task() {
    action0
    action1
    ...
}
```

- Functions use the sh shell syntax, with available OpenEmbedded variables and internal functions available.
  - WORKDIR: the recipe's working directory
  - UNPACKDIR: The directory where fetched sources are placed
  - S: The directory where the source code is extracted
  - B: The directory where `bitbake` places the objects generated during the build
  - D: The destination directory (root directory of where the files are installed, before creating the image).



Writing tasks 2/2

- Example:

```
do_compile() {
    oe_runmake
}

do_install() {
    install -d ${D}${bindir}
    install -m 0755 hello ${D}${bindir}
}
```



Adding new tasks

- Tasks can be added with `addtask`

```
do_mkimage () {
    uboot-mkimage ...
}

addtask do_mkimage after do_compile before do_install
```

- Tasks are commonly added by classes



Applying patches



Patch use cases

Patches can be applied to resolve build-system problematics:

- To support old versions of a software: bug and security fixes.
- To fix cross-compilation issues.
- To apply patches before they make their way into the upstream version.

However, there are cases when patching a `Makefile` is unnecessary:

- For example, when an upstream `Makefile` uses hardcoded `CC` and/or `CFLAGS`.
- You can call `make` with the `-e` option which gives precedence to variables taken from the environment:

```
EXTRA_OEMAKE = "-e"
```



The source locations: patches

- Files ending in `.patch`, `.diff` or having the `apply=yes` parameter will be applied after the sources are retrieved and extracted, during the `do_patch` task.
  - Compressed patches with `.gz`, `.bz2`, `.xz` or `.Z` suffix are automatically decompressed

```
SRC_URI += "file://joystick-support.patch \
            file://smp-fixes.diff \
           "
```

- Patches are applied in the order they are listed in SRC_URI.
- It is possible to select which tool will be used to apply the patches listed in SRC_URI variable with PATCHTOOL.
- By default, `PATCHTOOL = "quilt"` in openembedded-core.
- Possible values: `git`, `patch` and `quilt`.



Resolving conflicts

- The PATCHRESOLVE variable defines how to handle conflicts when applying patches.
- It has two valid values:
  - `noop`: the build fails if a patch cannot be successfully applied.
  - `user`: a shell is launched to resolve manually the conflicts.
- By default, `PATCHRESOLVE = "noop"` in `openembedded-core`.



Example of a recipe



Hello world recipe

```
SUMMARY = "Hello world program"
DESCRIPTION = "Hello world program"
HOMEPAGE = "http://example.net/hello/"
SECTION = "examples"
LICENSE = "GPL-2.0-or-later"

SRC_URI = "git://git.example.com/hello;protocol=https;branch=master"
SRCREV = "2d47b4eb66e705458a17622c2e09367300a7b118"
LIC_FILES_CHKSUM = "file://hello.c;beginline=3;endline=21;md5=58e..."

do_compile() {
    oe_runmake
}
do_install() {
    install -d ${D}${bindir}
    install -m 0755 hello ${D}${bindir}
}
```



Example of a recipe with a version agnostic part



tar.inc

```
SUMMARY = "GNU file archiving program"
HOMEPAGE = "https://www.gnu.org/software/tar/"
SECTION = "base"

SRC_URI = "${GNU_MIRROR}/tar/tar-${PV}.tar.bz2"

do_configure() { ... }

do_compile() { ... }

do_install() { ... }
```



tar_1.17.bb

```
require tar.inc

LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=59530bdf33659b29e73d4adb9f9f6552"

SRC_URI += "file://avoid_heap_overflow.patch"

SRC_URI[sha256sum] = "cec18d7f18fe5..."
```



tar_1.26.bb

```
require tar.inc

LICENSE = "GPL-3.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=d32239bcb673463ab874e80d47fae504"

SRC_URI[sha256sum] = "731140004fdb6..."
```



Debugging recipes



Log and run files

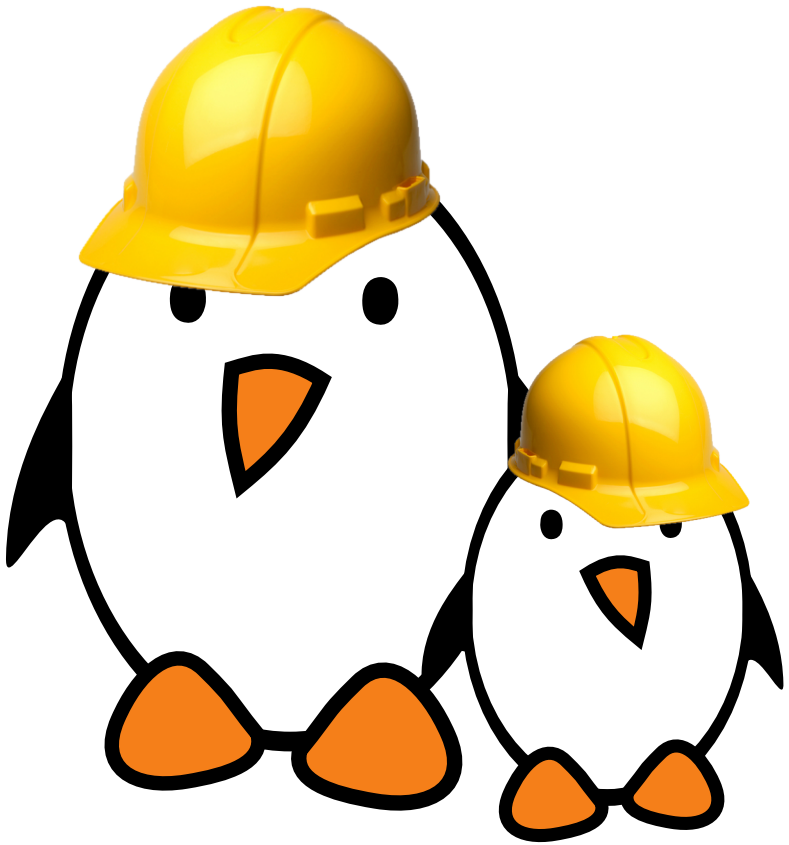
- For each task, these files are generated in the `temp` directory under the recipe work directory
- `run.do_<taskname>`
  - the script generated from the recipe content and executed to run the task
- `log.do_<taskname>`
  - the output of the task execution
- These can be inspected to understand what is being done by the tasks



Debugging variable assignments

- (Reminder) `bitbake-getvar` can dump the per-recipe variable value using the `-r` option
  - `bitbake-getvar -r ncurses SRC_URI`
- Similarly, `bitbake -e` dumps the entire environment, and also the task code
  - `bitbake -e` (**global** environment)
  - `bitbake -e ncurses` (environment of `ncurses`)

Practical lab - Add a custom application



- Write a recipe for a custom application
- Integrate it in the image